Name: Mayuresh Mali

PRN: 202501090018

Division: ME1

Batch: M11

Course: EDS

Topic:Amazon product dataset

Dataset Description (Amazon Product Dataset)
The Amazon Product Dataset contains information about various products listed on Amazon. Each row represents a single product, and the dataset includes important attributes related to pricing, customer feedback, and product categorization.
Key Features:
Product Name – Name/title of the product
Category – Type or group the product belongs to
Price – Cost of the product
Rating – Average customer rating (usually out of 5)
Reviews – Number of customer reviews
Nature of Data:
Combination of numerical data (price, rating, reviews)
Categorical data (product category, name)
Purpose:
This dataset is used to analyze product performance, pricing trends, customer satisfaction, and category-wise distribution of products on an e-commerce platform.

Here's a step-by-step explanation of the notebook:

1. Importing Libraries & Loading Data
   - pandas is imported for data manipulation and numpy for numerical operations
   - The dataset is loaded from a CSV file into a DataFrame called df
2. Previewing the Data
   - df.head() prints the first 5 rows to get a quick look at the structure — columns include product id, brand, categories, review text, ratings, usernames, etc.
3. Basic Statistics
   - avg_rating — calculates the mean of the reviews.rating column to find the overall average rating
   - total_reviews — counts the total number of rows (i.e., total reviews) using len(df)
4. Reviews Per Product
   - reviews_per_product — uses value_counts() on the name column to count how many reviews each product has received
5. Highest & Lowest Rated Reviews
   - highest_review — filters rows where the rating equals the maximum rating (5 stars)
   - lowest_review — filters rows where the rating equals the minimum rating (1 star)

6. Recommendation Analysis

- recommend_count — sums the reviews.doRecommend column (True = 1) to count how many reviewers recommended the product
- not_recommend — subtracts the recommend count from total rows to find how many did NOT recommend

7. Helpfulness Analysis

- avg_helpful — finds the average number of helpful votes across all reviews
- most_helpful — finds the single review that received the highest number of helpful votes using idxmax()

8. Top Products & Average Ratings

- top_products — lists the 5 most reviewed products
- avg_rating_product — groups by product name and computes the average rating per product
- avg_rating_brand — groups by brand and computes the average rating per brand
- reviews_per_brand — counts total reviews per brand

9. Rating Distribution

- five_star_percent — calculates the percentage of reviews that are exactly 5 stars
- low_rating_percent — calculates the percentage of reviews with ratings of 2 or below

10. High Performing Products

- high_products — groups by product name, computes average ratings, then filters only those products with an average rating above 4

11. Duplicate Detection

- duplicates — identifies rows where the same username has reviewed the same product more than once, which could indicate spam or repeated submissions

12. Most Active Reviewer

top_user — finds the username who has written the most reviews using value_counts().idxmax()

13. Reviews Per Year

The dateAdded column is converted to a datetime format (timezone-aware)

A new year column is extracted from it

reviews_per_year counts how many reviews were added each year — output shows 2017 had the most (872), followed by 2015 (524) and 2016 (201)

14. Outlier Detection

The IQR (Interquartile Range) method is used to detect rating outliers

Q1 (25th percentile) and Q3 (75th percentile) are calculated, and IQR = Q3 - Q1

Reviews with ratings below Q1 - 1.5×IQR or above Q3 + 1.5×IQR are flagged as outliers

CODE

```python
import pandas as pd
import numpy as np

df = pd.read_csv("/content/7817_1.csv")
```

```python
print(df.head())
```

```
...                     id       asins   brand                  categories  \
0  AVpe7AsMilAPnD_xQ78G  B00QJDU3KY  Amazon  Amazon Devices,mazon.co.uk
1  AVpe7AsMilAPnD_xQ78G  B00QJDU3KY  Amazon  Amazon Devices,mazon.co.uk
2  AVpe7AsMilAPnD_xQ78G  B00QJDU3KY  Amazon  Amazon Devices,mazon.co.uk
3  AVpe7AsMilAPnD_xQ78G  B00QJDU3KY  Amazon  Amazon Devices,mazon.co.uk
4  AVpe7AsMilAPnD_xQ78G  B00QJDU3KY  Amazon  Amazon Devices,mazon.co.uk

  colors            dateAdded          dateUpdated  \
0    NaN  2016-03-08T20:21:53Z  2017-07-18T23:52:58Z
1    NaN  2016-03-08T20:21:53Z  2017-07-18T23:52:58Z
2    NaN  2016-03-08T20:21:53Z  2017-07-18T23:52:58Z
3    NaN  2016-03-08T20:21:53Z  2017-07-18T23:52:58Z
4    NaN  2016-03-08T20:21:53Z  2017-07-18T23:52:58Z

                dimension  ean                          keys  ...  \
0  169 mm x 117 mm x 9.1 mm  NaN  kindlepaperwhite/b00qjdu3ky  ...
1  169 mm x 117 mm x 9.1 mm  NaN  kindlepaperwhite/b00qjdu3ky  ...
2  169 mm x 117 mm x 9.1 mm  NaN  kindlepaperwhite/b00qjdu3ky  ...
3  169 mm x 117 mm x 9.1 mm  NaN  kindlepaperwhite/b00qjdu3ky  ...
4  169 mm x 117 mm x 9.1 mm  NaN  kindlepaperwhite/b00qjdu3ky  ...

  reviews.rating                          reviews.sourceURLs  \
0            5.0  https://www.amazon.com/Kindle-Paperwhite-High:...
1            5.0  https://www.amazon.com/Kindle-Paperwhite-High:...
2            4.0  https://www.amazon.com/Kindle-Paperwhite-High:...
...
```

```
...
3                NaN            Dougal    NaN NaN  205 grams
4                NaN  Miljan David Tanic    NaN NaN  205 grams

[5 rows x 27 columns]
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

```python
avg_rating = df['reviews.rating'].mean()
```

```python
total_reviews = len(df)
```

```python
reviews_per_product = df['name'].value_counts()
```

```python
highest_review = df[df['reviews.rating'] == df['reviews.rating'].max()]
```

```python
lowest_review = df[df['reviews.rating'] == df['reviews.rating'].min()]
```

```python
recommend_count = df['reviews.doRecommend'].sum()
```

```python
not_recommend = len(df) - df['reviews.doRecommend'].sum()
```

```python
not_recommend = len(df) - df['reviews.doRecommend'].sum()
```

```python
avg_helpful = df['reviews.numHelpful'].mean()
```

```python
most_helpful = df.loc[df['reviews.numHelpful'].idxmax()]
```

```python
top_products = df['name'].value_counts().head(5)
```

```python
avg_rating_product = df.groupby('name')['reviews.rating'].mean()
```

```python
avg_rating_brand = df.groupby('brand')['reviews.rating'].mean()
```

```python
reviews_per_brand = df['brand'].value_counts()
```

```python
five_star_percent = (np.sum(df['reviews.rating'] == 5) / len(df)) * 100
```

```python
low_rating_percent = (np.sum(df['reviews.rating'] <= 2) / len(df)) * 100
```

```python
high_products = df.groupby('name')['reviews.rating'].mean()
high_products = high_products[high_products > 4]
```

```python
duplicates = df[df.duplicated(subset=['reviews.username', 'name'])]
```

```python
top_user = df['reviews.username'].value_counts().idxmax()
```

```python
# Use 'dateAdded' instead of 'reviews.date'
df['dateAdded'] = pd.to_datetime(df['dateAdded'], utc=True)
df['year'] = df['dateAdded'].dt.year

reviews_per_year = df['year'].value_counts()
print(reviews_per_year)
```

```
year
2017    872
2015    524
2016    201
Name: count, dtype: int64
```

```python
q1 = np.percentile(df['reviews.rating'], 25)
```


```python
high_products = df.groupby('name')['reviews.rating'].mean()
high_products = high_products[high_products > 4]
```

```python
duplicates = df[df.duplicated(subset=['reviews.username', 'name'])]
```

```python
top_user = df['reviews.username'].value_counts().idxmax()
```

```python
# Use 'dateAdded' instead of 'reviews.date'
df['dateAdded'] = pd.to_datetime(df['dateAdded'], utc=True)
df['year'] = df['dateAdded'].dt.year

reviews_per_year = df['year'].value_counts()
print(reviews_per_year)
```

```
year
2017    872
2015    524
2016    201
Name: count, dtype: int64
```


```python
q1 = np.percentile(df['reviews.rating'], 25)
q3 = np.percentile(df['reviews.rating'], 75)
iqr = q3 - q1

outliers = df[(df['reviews.rating'] < q1 - 1.5*iqr) |
              (df['reviews.rating'] > q3 + 1.5*iqr)]
```

Conclusion

The Amazon dataset analysis demonstrates how product data can reveal customer behavior, pricing strategies, and product performance. NumPy and Pandas provide efficient tools for analyzing large-scale e-commerce datasets.

LINK FOR DATASET

https://drive.google.com/file/d/1PWYlM3Rsf-CPB1DEVYXEirpdiL5n4LpC/view?usp=sharing